

Project Title:

Project k - Club Portfolio Portal

Version: 1.0

Prepared By: **Oyster Kode Club Project Team**

Project Type: Long-Term Club Deployment

Document Purpose: Internal Database Design Specification and
Implementation Planning

1. Introduction

This document describes the complete database architecture for Project K – Club Portfolio Portal. The database is designed to support role-based access, profile management, skill filtering, resume downloads, analytics tracking, and administrative governance.

2. Technology Stack

- Database: PostgreSQL (via Supabase)
- Authentication: Supabase Auth
- Storage: Supabase Storage (Resumes Bucket)
- Security: Row Level Security (RLS) Policies

3. User Roles

- General Visitor (Pre-login Public User)
- Club Member (Approved Active Member)
- Club Alumni (Graduated Member with Alumni Tag)
- Recruiter (External Professional)
- Guest (Authenticated General User)
- Admin (Platform Authority)

4. Database Objectives

- Maintain structured relational integrity
- Enforce role-based permissions
- Ensure secure resume handling
- Support skill-based search and filtering
- Enable analytics monitoring
- Support admin approval system

5. Core Database Tables

- profiles
- contact_info

- skills
- member_skills
- projects
- resumes
- profile_views
- resume_downloads
- approval_requests
- audit_logs

Core Tables and Schemas

Table: profiles

- id (UUID, PK)
- full_name (TEXT)
- YearOfGraduation(INTEGER)
- email (TEXT)
- role (TEXT)
- bio (TEXT)
- tagline (TEXT)
- availability (TEXT)
- is_approved (BOOLEAN)
- created_at (TIMESTAMP)

Table: contact_info

- user_id (UUID, PK, FK -> profiles.id)
- phone (TEXT)
- linkedin (TEXT)
- github (TEXT)
- portfolio_url (TEXT)

Table: skills

- id (UUID, PK)
- name (TEXT)
- category (TEXT)

Table: member_skills

- user_id (UUID, FK -> profiles.id)
- skill_id (UUID, FK -> skills.id)
- level (TEXT)
- Primary Key (user_id, skill_id)

Table: projects

- id (UUID, PK)
- user_id (UUID, FK -> profiles.id)
- title (TEXT)
- description (TEXT)
- github_link (TEXT)
- tech_stack (TEXT)
- created_at (TIMESTAMP)

Table: resumes

- id (UUID, PK)
- user_id (UUID, FK -> profiles.id)
- file_path (TEXT)
- uploaded_at (TIMESTAMP)

Table: profile_views

- id (UUID, PK)
- viewed_profile_id (UUID, FK -> profiles.id)

- viewer_id (UUID, FK -> profiles.id)
- timestamp (TIMESTAMP)

Table: resume_downloads

- id (UUID, PK)
- profile_id (UUID, FK -> profiles.id)
- downloader_id (UUID, FK -> profiles.id)
- timestamp (TIMESTAMP)

Table: approval_requests

- id (UUID, PK)
- user_id (UUID, FK -> profiles.id)
- status (TEXT)
- reviewed_by (UUID, FK -> profiles.id)
- reviewed_at (TIMESTAMP)

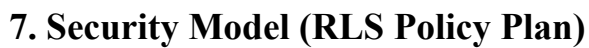
Table: audit_logs

- id (UUID, PK)
- user_id (UUID, FK -> profiles.id)
- action (TEXT)
- timestamp (TIMESTAMP)

5. Relationship Overview

- profiles ↔ contact_info (One-to-One)
- profiles ↔ projects (One-to-Many)
- profiles ↔ skills (Many-to-Many via member_skills)
- profiles ↔ resumes (One-to-Many)
- profiles ↔ profile_views (One-to-Many)
- profiles ↔ resume_downloads (One-to-Many)

- ## 6. Logical Database Architecture Diagram



- Members can edit only their own profile
- Public users cannot view contact information
- Only authenticated users can download resumes
- Recruiters can view detailed profiles and contact info
- Admins can approve members and monitor analytics
- Platform authority restricted to Admin role only

8. Authentication & Authorization Model

The system uses Google OAuth 2.0 authentication integrated via Supabase Auth.

Users authenticate using their Google accounts, and upon successful login, Supabase generates a unique user identifier (UUID). This UUID is used as the primary reference key in the profiles table and related tables.

No passwords are stored within the application database, ensuring enhanced security and compliance with modern authentication standards.

9. Storage Design

Bucket Name: resumes

Allowed File Type: PDF

Maximum Size: 5MB

Access: Authenticated Users Only

10. Indexing Strategy

- Index on profiles.role
- Index on profiles.availability
- Index on member_skills.skill_id
- Index on projects.user_id
- Index on profile_views.viewed_profile_id

11. Conclusion

This database architecture ensures scalability, security, and performance while fully aligning with the SRS role-permission matrix. The schema is normalized, relationally structured, and designed to support future feature expansion.